

SciFiGame

Technical Architecture & API Reference

1. Project Overview

SciFiGame is a third-person corridor-stealth action game built on Unity 2022+ with the New Input System and Cinemachine. The entire codebase is organized around one central principle: **systems never hold direct references to one another**. All cross-system communication travels through a static event bus (`GameEvents`). This makes every system independently testable, hot-swappable, and extensible without modifying existing code.

Namespace Structure

Namespace	Purpose
<code>SciFiGame.Core</code>	Event bus, payload structs, PlayerState enum, PlayerStateMachine
<code>SciFiGame.Input</code>	InputReader ScriptableObject, InputConstants
<code>SciFiGame.Camera</code>	CameraController, CameraSettingsData
<code>SciFiGame.Player</code>	Movement, animation, footsteps, rotation, input aggregation
<code>SciFiGame.Laser</code>	LaserBeam, LaserBeamVisual, LaserMovementComponent, hit response
<code>SciFiGame.QTE</code>	QTEManager, QTESequenceData, QTETriggerZone, UI widgets, types
<code>SciFiGame.Checkpoint</code>	Checkpoint trigger, CheckpointManager, respawn logic
<code>SciFiGame.Timeline</code>	TimelineController adapter, GlobalTimelinePlayer
<code>SciFiGame.Interaction</code>	IInteractable interface, EndDoorComponent, StealableArticleComponent
<code>SciFiGame.UI</code>	GameOver, EscapeMenu, InteractionPrompt, Tutorial, QTE widgets

<code>SciFiGame.Audio</code>	AudioManagerComponent singleton
<code>SciFiGame.Environment</code>	TutorialTriggerComponent

Core Architecture Principles

- **Event Bus over Direct References:** All cross-system communication goes through `GameEvents` static events. No component holds a `MonoBehaviour` reference to another unrelated system.
- **ScriptableObject-Driven Data:** Movement tuning, camera profiles, and QTE sequences all live in assets. Designers can tweak and create new variants without touching C# code.
- **Single-Responsibility Components:** The player is not one fat `PlayerController`. Input reading, movement, animation, rotation, and footsteps are each separate components that share data through properties.
- **LateUpdate for Camera:** `CameraController` runs in `LateUpdate` so all movement has committed for the frame before the camera applies its rotation — prevents one-frame jitter.
- **Struct Payloads, Zero GC Allocation:** All event payloads are readonly struct value types. Raising an event does not allocate on the heap.
- **Invincible Layer Pattern:** During QTEs, death animations, and cinematic timelines, the player's physics layer switches to "Invincible", preventing laser collisions mid-cutscape.

2. Event Bus — GameEvents

File: `GameEvents.cs` | Namespace: `SciFiGame.Core`

`GameEvents` is a static class — no manager `GameObject`, no `FindObjectOfType`, no singleton boilerplate. Any script anywhere can raise or subscribe to an event.

Design Rationale: All events use null-conditional raise helpers (`RaiseXxx` methods) so call sites never write `OnXxx?.Invoke()` directly. Payload structs carry exactly the data subscribers need and are readonly to prevent mutation.

Full Event Catalogue

Event	Payload Type	Raised By	Consumed By
<code>OnPlayerStateChanged</code>	<code>PlayerStateChangedPayload</code>	<code>PlayerStateMachine</code>	Any system needing state awareness

OnPlayerDied	PlayerDeathPayload	LaserHitResponseController, QTEManager	CheckpointManager, CameraController, PlayerInputComponent, RotationComponent, PlayerStateMachine
OnPlayerRespawned	CheckpointPayload	CheckpointManager	LaserBeam, QTETriggerZone, PlayerInputComponent, CameraController, RotationComponent, PlayerStateMachine
OnPlayerEnteredQTEZone	QTEZonePayload	QTETriggerZone	QTEManager
OnQTEStarted	QTEZonePayload	QTEManager	CameraController, PlayerInputComponent, RotationComponent, PlayerStateMachine, all QTE UI widgets
OnQTEInputReceived	QTEInputPayload	QTEManager	QTETimerWidget, SequenceWidget
OnQTESucceeded	QTEZonePayload	QTEManager	CameraController, PlayerInputComponent, RotationComponent, PlayerStateMachine, QTETriggerZone, QTE UI widgets
OnQTEFailed	QTEZonePayload	QTEManager	QTE UI widgets
OnTimelineStarted	TimelinePayload	TimelineController	PlayerStateMachine
OnTimelineFinished	TimelinePayload	TimelineController	PlayerStateMachine
OnCheckpointActivated	CheckpointPayload	Checkpoint	CheckpointManager
OnGameOver	(none)	EndDoorComponent	GameOverUIComponent
OnInteractionPromptChanged	InteractionPromptPayload	PlayerMovementComponent	InteractionUIComponent
OnTutorialPromptTriggered	TutorialPromptPayload	TutorialTriggerComponent	TutorialUIComponent
OnLaserEntered / Exited	(none)	laser zone boundary	PlayerStateMachine

Correct Usage Pattern

```
private void OnEnable() { GameEvents.OnPlayerDied += HandleDeath; }
private void OnDisable() { GameEvents.OnPlayerDied -= HandleDeath; }
private void HandleDeath(PlayerDeathPayload p) { /* react */ }

// Raising an event:
GameEvents.RaisePlayerDied(new PlayerDeathPayload(myTimelineAsset));
```

3. Input System

File: `InputReader.cs` | Namespace: `SciFiGame.Input`

`InputReader` is a ScriptableObject wrapping Unity's New Input System. Components subscribe to its typed C# events rather than polling `Input.GetKey` or calling `FindAction` at runtime.

Gameplay Events

Event	C# Type	Description
<code>OnMoveInput</code>	Action<Vector2>	WASD / left-stick composite. Fires on performed and canceled.
<code>OnLookInput</code>	Action<Vector2>	Mouse delta / right-stick. Fires on performed.
<code>OnCrouchInput</code>	Action<bool>	true on press (started), false on release (canceled).
<code>OnInteractInput</code>	Action	Fires once on started.
<code>OnPauseInput</code>	Action	Escape key started.
<code>OnResumeInput</code>	Action	UI Cancel action started.

Map-Switching API

Method	Enables	Disables
<code>EnableGameplay()</code>	Gameplay	QTE, UI
<code>EnableQTE()</code>	QTE	Gameplay, UI

<code>EnableUI()</code>	UI	Gameplay, QTE
<code>DisableAllMaps()</code>	—	Gameplay, QTE, UI

4. Camera System

File: `CameraController.cs` | Namespace: `SciFiGame.Camera`

Drives the `CameraRoot` Transform for yaw (horizontal) and pitch (vertical) rotation. Cinemachine's Third Person Follow tracks `CameraRoot`, not the player body, so the game gets full over-shoulder control without fighting Cinemachine's internal position logic.

Inspector Parameters

Field	Default	Description
<code>_cameraRootTransform</code>	—	The Transform Cinemachine tracks. This is what the game rotates.
<code>_horizontalSensitivity</code>	0.2	Mouse X multiplier.
<code>_verticalSensitivity</code>	0.2	Mouse Y multiplier.
<code>_pitchMax</code> / <code>_pitchMin</code>	20° / -35°	Upper and lower pitch limits.
<code>_crouchDropOffset</code>	0.5	Metres the camera root descends at full crouch.

5. Player System

The player uses a composition model: six focused MonoBehaviour components on the player hierarchy, each with a single job.

- **PlayerStateMachine:** Evaluates and transitions between `PlayerState` values each Update. Manages the Invincible physics layer.
- **PlayerMovementComponent:** Drives CharacterController velocity using data from a `MovementData` ScriptableObject.
- **PlayerInputComponent:** The only component that knows about `InputReader`. Stores a frame-coherent snapshot.
- **PlayerAnimationComponent:** Translates movement state into Animator parameters.
- **PlayerFootstepComponent:** Fires footstep audio on a timer synchronized to movement state.
- **RotationComponent:** Rotates the character body toward `WorldVelocity`.

6. Laser System

The laser system separates four distinct concerns across four components:

- **LaserBeamVisual:** Renders the laser and automatically resizes a BoxCollider to match the visual beam length (stops at walls).
- **LaserBeam:** Handles collision detection only. Raises `OnLaserHit`.
- **LaserHitResponseController:** Determines the consequence (kills the player).
- **LaserMovementComponent:** Handles transform animation (oscillation, rotation) and flashing behaviour.

7. Quick-Time Event (QTE) System

File: `QTEManager.cs` | Namespace: `SciFiGame.QTE`

The QTE system is built from data assets, trigger zones, an orchestrator, and UI widgets. `QTEManager` orchestrates the full QTE lifecycle. It is a scene-present MonoBehaviour, not a singleton.

QTEInputType Enum

Value	Player Action	Success Condition
<code>SinglePress</code>	Tap the correct key once.	Matching ActionName received while QTE is active.
<code>Hold</code>	Hold the key.	Held for at least <code>TimeLimit × HoldTolerance</code> seconds.
<code>Mash</code>	Press the key rapidly.	<code>MashThreshold</code> presses before timer expires.
<code>Sequence</code>	Press keys in a specific order.	All <code>Steps[]</code> pressed in correct sequence before timeout.

8. Checkpoint & Respawn System

Checkpoint: A passive trigger volume. Stores no state. Raises `OnCheckpointActivated`.

CheckpointManager: Owns all checkpoint state. When `OnPlayerDied` fires, it teleports the player to the last checkpoint's position and plays the death/respawn timeline.

9. Timeline System

TimelineController: An adapter component attached to PlayableDirectors. Relays director events as GameEvents (`OnTimelineStarted` , `OnTimelineFinished`).

GlobalTimelinePlayer: Used by CheckpointManager to play a TimelineAsset at runtime.

PlayerInputDisableTrack: A custom Timeline track that safely suspends and restores input during cinematics.

10. Interaction System

Any world object the player can interact with implements `IInteractable`. `PlayerMovementComponent` raycasts for it and calls `GetInteractionPrompt()` to drive the HUD, and `Interact()` when triggered.

```
public interface IInteractable
{
    void Interact();
    string GetInteractionPrompt();
}
```

11. Audio System

AudioManagerComponent: Singleton MonoBehaviour for global 2D sounds. Exposes `PlayOneShot(AudioClip clip, float volumeScale = 1f)`.

12. UI System

- **EscapeMenuUIComponent:** Handles pause/resume logic and time scaling.
- **GameOverUIComponent:** Displays game over screen and handles scene reload/quit.
- **TutorialUIComponent:** Displays auto-hiding tutorial messages driven by `TutorialTriggerComponent`.

13. Cross-System Event Dependency Matrix

Component	OnPlayerDied	OnPlayerRespawned	OnQTEShared	OnQTESucceeded
PlayerStateMachine	S	S	S	S
PlayerInputComponent	S	S	S	S
CameraController	S	S	S	S
RotationComponent	S	S	S	S
CheckpointManager	S	-	-	-

QTEManager	-	-	-	-
QTE UI Widgets	-	-	S	S

14. Known Issues & Technical Debt

Location	Issue	Priority
CheckpointManager	No null guard if OnPlayerDied fires before any checkpoint is ever activated.	High
QTEManager	OnQTEFailed is commented out in multiple OnDisable unsubscription blocks.	Medium
StealableArticle	Interact() needs inventory update logic or quest flag integration.	Medium
EscapeMenu	Hard-codes scene index 0 for the main menu.	Low